

SQL Assignment — Group 8: Transactions

COMMIT ROLLBACK SAVEPOINT and Triggers

Group 8 — Transactions (COMMIT, ROLLBACK, SAVEPOINT) & Triggers

1. INSERT + COMMIT, verify persistence

Insert a new order into an Orders table, then use COMMIT to save the transaction and verify it persists after reconnecting.

```
INSERT INTO Orders (order_id, user_id, order_total, order_date)
VALUES (501, 12, 899.00, CURRENT_DATE);

COMMIT;

-- After disconnecting and reconnecting to the database, this query
-- should still return the row, confirming the COMMIT made it permanent:
SELECT * FROM Orders WHERE order_id = 501;
```

Explanation: Until a transaction is committed, its changes are typically only visible within that same session and can still be undone. `COMMIT;` permanently saves the `INSERT` to the database, so even after closing the connection and reconnecting later, the new order (order_id 501) will still be present when queried.

2. Zomato-style order: INSERT then ROLLBACK

Insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain after rollback.

```
INSERT INTO OrderItems (item_id, order_id, item_name, quantity)
VALUES (301, 501, 'Paneer Tikka', 2);

INSERT INTO OrderItems (item_id, order_id, item_name, quantity)
VALUES (302, 501, 'Butter Naan', 4);

ROLLBACK;

-- This should return zero rows, since the ROLLBACK undid both inserts
-- before they were committed:
SELECT * FROM OrderItems WHERE order_id = 501;
```

Explanation: Both `INSERT` statements happen inside the same uncommitted transaction. `ROLLBACK;` reverses every change made since the transaction began (or since the last `COMMIT`), so neither "Paneer Tikka" nor "Butter Naan" is actually saved — the follow-up `SELECT` confirms this by returning no rows for that order.

3. SAVEPOINT and partial ROLLBACK in a Cart

Use SAVEPOINT after adding a product to the Cart table, add another product, then ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.

```
INSERT INTO Cart (cart_item_id, user_id, product_name)
```

```
VALUES (1, 7, 'Wireless Mouse');

SAVEPOINT after_first_item;

INSERT INTO Cart (cart_item_id, user_id, product_name)
VALUES (2, 7, 'Laptop Stand');

ROLLBACK TO SAVEPOINT after_first_item;

COMMIT;

-- Final contents of the Cart table for this user:
SELECT * FROM Cart WHERE user_id = 7;
-- Expected result: only "Wireless Mouse" remains; "Laptop Stand" was undone.
```

Explanation: `SAVEPOINT after_first_item;` marks a checkpoint within the transaction, right after the first product is added. The second product ("Laptop Stand") is then inserted, but `ROLLBACK TO SAVEPOINT after_first_item;` undoes only the changes made after that checkpoint — the second insert — while keeping the first insert intact. The final `COMMIT` then permanently saves the transaction in its rolled-back state, so only "Wireless Mouse" remains in the cart.

4. Trigger to deduct Wallet balance on transaction insert

Write a trigger on a `Wallet` table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to Paytm.

```
-- Assumes: Wallet(user_id, balance) and Transactions(transaction_id, user_id, amount)

DELIMITER //

CREATE TRIGGER deduct_wallet_balance
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END//

DELIMITER ;
```

Explanation: This trigger fires automatically `AFTER INSERT ON Transactions` — every time a new row is added to the `Transactions` table. `NEW.amount` and `NEW.user_id` refer to the values from the row that was just inserted; the trigger uses them to subtract the transaction amount from that user's `balance` in the `Wallet` table, keeping the wallet automatically in sync without the application needing a separate manual update statement.

5. Buggy trigger scenario and correction

Generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining the fix.

```
-- ===== BUGGY VERSION =====
-- Bug: adds the expense amount instead of subtracting it, and
-- doesn't check that the user actually has enough balance.

DELIMITER //

CREATE TRIGGER deduct_wallet_balance_buggy
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance + NEW.amount -- BUG: should be '-', not '+'
```

```

        WHERE user_id = NEW.user_id;
    END//

DELIMITER ;

-- TEST CASE THAT EXPOSES THE BUG:
-- Assume Wallet has (user_id=7, balance=500)
INSERT INTO Transactions (transaction_id, user_id, amount) VALUES (901, 7, 200);
-- Expected balance after this expense: 500 - 200 = 300
-- Actual balance with the buggy trigger: 500 + 200 = 700 (WRONG — balance went UP after a purchase)

-- ===== CORRECTED VERSION =====
DELIMITER //

CREATE TRIGGER deduct_wallet_balance_fixed
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END//

DELIMITER ;

-- Re-running the same test case with the fixed trigger now correctly
-- reduces the balance from 500 to 300 after the 200 expense.

```

Explanation: The bug is a single-character sign error: `balance + NEW.amount` instead of `balance - NEW.amount`, which means every purchase would incorrectly *increase* the wallet balance rather than decrease it. The test case makes this obvious by tracking the expected balance (300) against what the buggy trigger actually produces (700). The fix simply changes the operator from `+` to `-`, restoring correct behavior — a more production-ready version would also add a check that `balance >= NEW.amount` before allowing the deduction, to prevent the balance from going negative.